

Discrete Mathematics and graph Theory

Chapter Five: Weighted graphs and their applications

Getachew Bitew

Department of Mathematics
Addis Ababa University

2024/25

Out lines

Topics

In this chapter we will discuss the following topics:

- 5.1.** Weighted graphs
- 5.2.** Minimal spanning trees
- 5.3.** Shortest path problem
- 5.4.** Critical path problem

Weighted graphs with examples

Weighted graphs are graphs where each edge has a numerical value (weight) representing cost, distance, or capacity, with common applications in routing, logistics, and resource allocation.

What do the weights Represent?

- ✉ Internet graph: Edge weights could correspond to the amount of data flowing along them or their bandwidth.
- ✉ Roadmap graph: Edge weights could correspond to lengths, numbers of gas stations.
- ✉ Food web graph: Edge weights could correspond to the total energy flow between prey and predator.
- ✉ Social network graph: Edge weights could correspond to the frequency of contact.

A weighted graph is a 3-tuple $G = (V, E, w)$, where V is the set of nodes, E is the set of edges, and $w : E \rightarrow \mathbb{R}$ ($\mathbb{R}$ is the set of reals) is a function that assigns a weight to each edge.

☞ The definition applies to both directed and undirected graphs.

Example

The following are weighted nondirected and directed graphs.

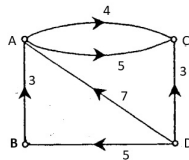
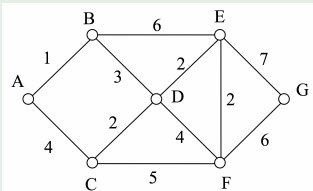


Figure: (a) Graph G, (b) Graph H

Trees, Spanning Trees, and Minimal Spanning Trees

Definition

- ✈ A tree is a connected, acyclic graph.
- ✈ A spanning tree of a graph $G(V, E)$ is a connected, acyclic subgraph of G that contains all the nodes in V .
- ✈ The weight of a spanning tree of a weighted graph $G(V, E, w)$ is the sum of the weights of the edges in the tree.
- ✈ A minimal spanning tree (MST) of a weighted graph $G(V, E, w)$ is a spanning tree whose weight is minimum among all possible spanning trees of G .

Problem formulation:

- Suppose we are given a connected, undirected, weighted graph $G = (V, E)$, together with a function $w : E \rightarrow \mathbb{R}$ that assigns a real weight $w(u, v)$ to each edge $(u, v) \in E$.
- Find an acyclic subset $T \subseteq E$ that connects all of the vertices V and minimizes the total weight:

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

In this regard, the minimum spanning tree is (V, T) .

Note: minimum spanning tree may not be unique.

Algorithm

Here we use algorithms to find the minimal spanning tree of G , that is, the spanning tree T that minimizes the function

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

MSTs are found most efficiently using algorithms like Kruskal's algorithm and Prim's algorithm. These algorithms work with undirected weighted graphs.

Kruskal's Algorithm for Shortest Spanning Trees

☞ Work with edges.

Given a connected graph $G = (V, E)$ with vertices $v_1, v_2, \dots, v_n$ and edges (v_i, v_j) having length $l_{ij} > 0$, the algorithm determines a shortest spanning tree T in G .

(INPUT) : Edges (v_i, v_j) of G and their lengths $l_{ij} > 0$

(OUTPUT) : Shortest spanning tree T in G

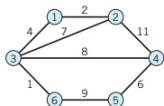
- 1 Order the edges of G in ascending order of length (weight).
- 2 Choose them in this order as edges of T , rejecting an edge only if it forms a cycle with edges already chosen.

If $n - 1$ edges have been chosen, then OUTPUT T (= the set of edges chosen). Stop

End KRUSKAL

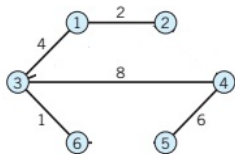
Example

Using Kruskal's algorithm, determine the minimum spanning tree for the following weighted graph.



Edges	Length/weight	Choice
(3, 6)	1	1 st
(1, 2)	2	2 nd
(1, 3)	4	3 rd
(4, 5)	6	4 th
(2, 3)	7	Reject
(3, 4)	8	5 th
(5, 6)	9	
(2, 4)	11	

We stop after $n - 1 = 5$ choices since a spanning tree has $n - 1$ edges. The minimum spanning tree is the following.



The minimum weight(length) of the spanning tree is $1 + 2 + 4 + 6 + 8 = 21$.

2. Prim's algorithm

- ☞ This algorithm avoids ordering edges and gives a tree T at each stage. In Prim's algorithm, we start with any single vertex.

1. ALGORITHM

Given a connected graph $G(V, E)$ with vertices $v_1, v_2, \dots, v_n$ and edges (v_i, v_j) having length $l_{ij} > 0$, this algorithm determines a shortest spanning tree T in G and its length $L(T)$.

INPUT: n , edges (v_i, v_j) of G and their lengths l_{ij} .

OUTPUT: Edge set S of a shortest spanning tree T in G ; $L(T)$.
[Initially, all vertices are unlabeled.]

- 1 Initial step: Set $i(k) = 1$, $U = \{v_1\}$, $S = \emptyset$.
Label vertex v_k ($k = 2, \dots, n$) with $\lambda_k = l_{1k}$ [$= \infty$ if G has no edge (v_1, v_k)].
- 2 Addition of an edge to the tree T . Let λ_j be the smallest λ_k for vertex v_k not in U . Include vertex v_j in U and edge $(v_{i(j)}, v_j)$ in S .

If $U = V$ then compute $L(T) = \sum l_{ij}$ (sum over all edges in S)
OUTPUT S , $L(T)$. Stop. [S is the edge set of a shortest spanning tree T in G .]

Else continue (that is, go to Step 3).

3. Label updating

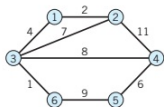
For every v_k not in U , if $l_{jk} < \lambda_k$, then set $\lambda_k = l_{jk}$
and $v_{i(k)} = v_j$.

Go to Step 2.

End PRIM.

Example

Using Prim's algorithm, determine the minimum spanning tree for the following weighted graph.

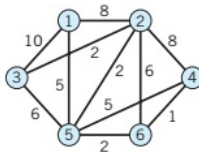
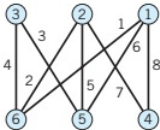
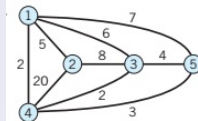
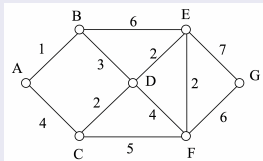


Solution

- ① Start with vertex 1. Hence $U = \{1\}$, and $S = \emptyset$.
- ② $\lambda_2 = w_{1,2} = 2$ is the smallest.
Hence, we have $U = \{1, 2\}$, and $S = \{(1, 2)\}$.
- ③ Update. $\lambda_3 = w_{1,3} = 4$ is the smallest.
Hence, we have $U = \{1, 2, 3\}$, and $S = \{(1, 2), (1, 3)\}$.
- ④ Update. $\lambda_6 = w_{3,6} = 1$ is the smallest.
Hence, we have $U = \{1, 2, 3, 6\}$ & $S = \{(1, 2), (1, 3), (3, 6)\}$.
- ⑤ Update. $\lambda_4 = w_{3,4} = 8$ is the smallest.
Hence, we have $U = \{1, 2, 3, 4, 6\}$, and
 $S = \{(1, 2), (1, 3), (3, 6), (3, 4)\}$.
- ⑥ Update. $\lambda_5 = w_{4,5} = 6$ is the smallest.
Hence, we have $U = \{1, 2, 3, 4, 5, 6\}$, and
 $S = \{(1, 2), (1, 3), (3, 6), (3, 4), (4, 5)\}$. Here we STOP.
The spanning tree is the same as the one we obtained using Kruskal's algorithm.

Exercise

Using Kruskal's algorithm and Prim's algorithm, determine the minimal spanning tree for the following weighted graph.



Shortest path problems

- ☞ The shortest path problem in a graph theory is finding the minimum-weight path between two nodes(vertices) in a graph, which determine the route with the lowest total cost(distance, time) from a source to all other nodes or a specific destination. Here we have two algorithms; Dijkstra's Algorithm and Bellman-Ford Algorithm.

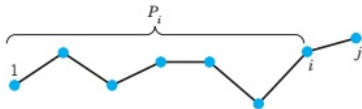
Given a weighted graph $G(V, V, w)$, such that each edge (v_i, v_j) has weight w_{ij} . The shortest path $v_1 \rightarrow v_k$ (with fixed v_1 and v_k) is a path of the graph such that the sum of the weights(lengths) of its edges

$$w_{1,2} + w_{2,3} + w_{3,4} + \cdots + w_{k-1,k}$$

is minimum.

Theorem (Bellman's Minimality Principle or Optimality Principle)

If $P_j : 1 \rightarrow j$ is a shortest path from 1 to j in G and (i, j) is the last edge of P_j (See the figure below), then $P_i : 1 \rightarrow i$ [obtained by dropping (i, j) from P_j] is a shortest path $1 \rightarrow i$.



Dijkstra's Algorithm

Dijkstra's Algorithm solves the single source shortest path problem for graphs with non negative edge weights.

- ✉ Initialization: Involves assigning a distance 0 to the source node and ∞ to other nodes.
- ✉ At any intermediate step, the algorithm divides the nodes into two groups, which are designated as Permanent label and Temporal label. Each vertex v gets either a permanent label (PL) or a temporary label (TL).

Note

- 1 *The distance label to any permanent node represent the shortest distance from the source to that node.*
- 2 *For any Temporal node, the distance label is an upper bound on the shortest path distance to that node.*

The general steps of the algorithm are given as follows.

ALGORITHM DIJKSTRA:

Given a connected graph $G = (V, E)$ with vertices $1, \dots, n$ and edges (i, j) having lengths $l_{ij} > 0$, this algorithm determines the lengths of shortest paths from vertex 1 to the vertices $2, \dots, n$.

INPUT: Number of vertices n , edges (i, j) , and length l_{ij} .

OUTPUT: Lengths L_j of shortest paths $1 \rightarrow j$, $j = 2, \dots, n$.

1. Initial step

Vertex 1 gets $PL : L_1 = 0$.

Vertex $j (= 2, \dots, n)$ gets $TL : \bar{L}_j = l_{1j}$ ($= \infty$ if there is no edge $(1, j)$ in G). Set $PL = \{1\}$,
 $TL = \{2, 3, \dots, n\}$.

2. Fixing a permanent label

Find a k in TL for which $\bar{L}_k$ is minimum, set $L_k = \bar{L}_k$. Take the smallest k if there are several. Delete k from TL and include it in PL . If $TL = \emptyset$ (that is, TL is empty) then OUTPUT $L_2, \dots, L_n$. Stop. Else continue (that is, go to Step 3).

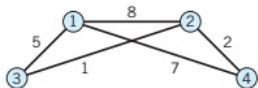
3. Updating temporary labels

For all j in TL, set $\bar{L}_j = \min\{\bar{L}_j, L_k + l_{kj}\}$ (that is, take the smaller of $\bar{L}_j$ and $L_k + l_{kj}$ as your new $\bar{L}_j$).
Go to Step 2.

End DIJKSTRA

Example

Using Dijkstra's algorithm to the graph below, find shortest paths from vertex 1 to vertices 2, 3, 4.



Solution

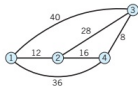
1.	$L_1 = 0, \bar{L}_2 = 8, \bar{L}_3 = 5, \bar{L}_4 = 7$	$PL = \{1\}$	$TL = \{2, 3, 4\}$
2.	$L_3 = \min\{\bar{L}_2, \bar{L}_3, \bar{L}_4\} = 5, k = 3$	$PL = \{1, 3\}$	$TL = \{2, 4\}$
3.	$L_2 = \min\{8, L_3 + l_{23}\} = \min\{8, 5 + 1\} = 6$		
	$\bar{L}_4 = \min\{7, L_3 + l_{34}\} = \min\{7, \infty\} = 7$		
2.	$L_2 = \min\{\bar{L}_2, \bar{L}_4\} = \min\{6, 7\} = 6, k = 2$	$PL = \{1, 2, 3\}$	$TL = \{4\}$
3.	$\bar{L}_4 = \min\{7, L_2 + l_{24}\} = \min\{7, 6 + 2\} = 7$		
2.	$L_4 = 7, k = 4$	$PL = \{1, 2, 3, 4\}$	$TL = \emptyset$

The resulting shortest paths, of lengths $L_2 = 6$, $L_3 = 5$, $L_4 = 7$. The shortest paths are depicted as follows.



Exercise

- 1 The net of roads in Figure below connecting four villages is to be reduced to minimum length, but so that one can still reach every village from every other village. Which of the roads should be retained? Find the solution (a) by inspection, (b) by Dijkstra's algorithm.



- 2 Using Dijkstra's algorithm, determine the shortest path for the following weighted graph.



Bellman-Ford Algorithm

Bellman-Ford Algorithm solves the single source shortest path problem for graphs with negative edge weights.

Algorithm: Reading Assignment.

END OF THE COURSE